Integration Design

How to choose the best pattern to integrate ServiceNow with other systems

Jochen Geist

Principal Platform Architect & ServiceNow Certified Master Architect
ServiceNow

servicenow®

Jochen Geist

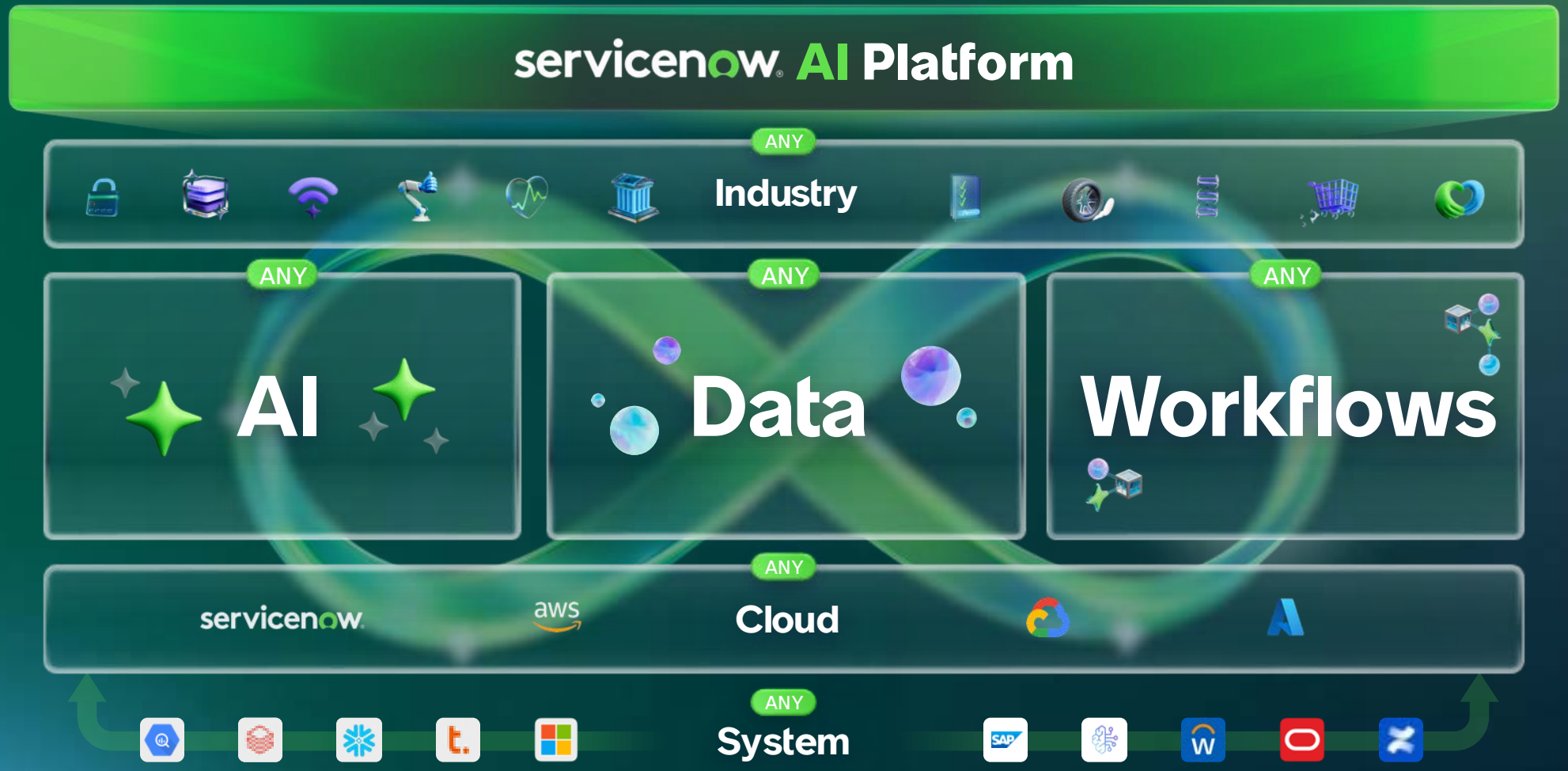
Principal Platform Architect
ServiceNow Certified Master Architect

ServiceNow

linkedin.com/in/jochengeist

Table of contents

One platform, ready for anything



Strategic Alignment

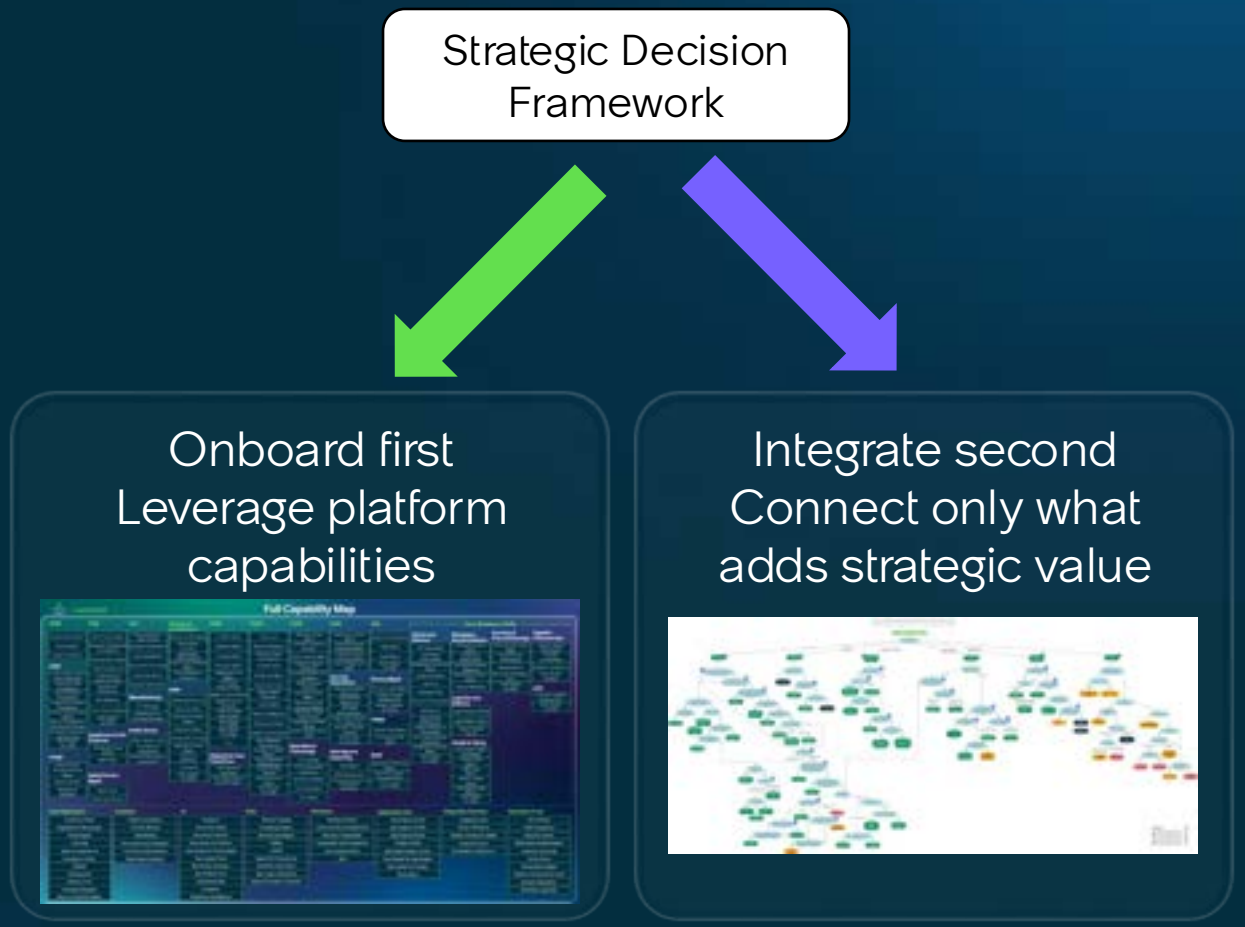
Business Value First

Core Principles:

- Every integration must support defined business objectives
- ➤ Does integration deliver more value than optimizing processes within ServiceNow?

Keys to Success:

- **Align stakeholder decisions** with enterprise platform strategy
- **Maximize ServiceNow capabilities** before adding integrations
- **Focus on business outcomes** that accelerate transformation



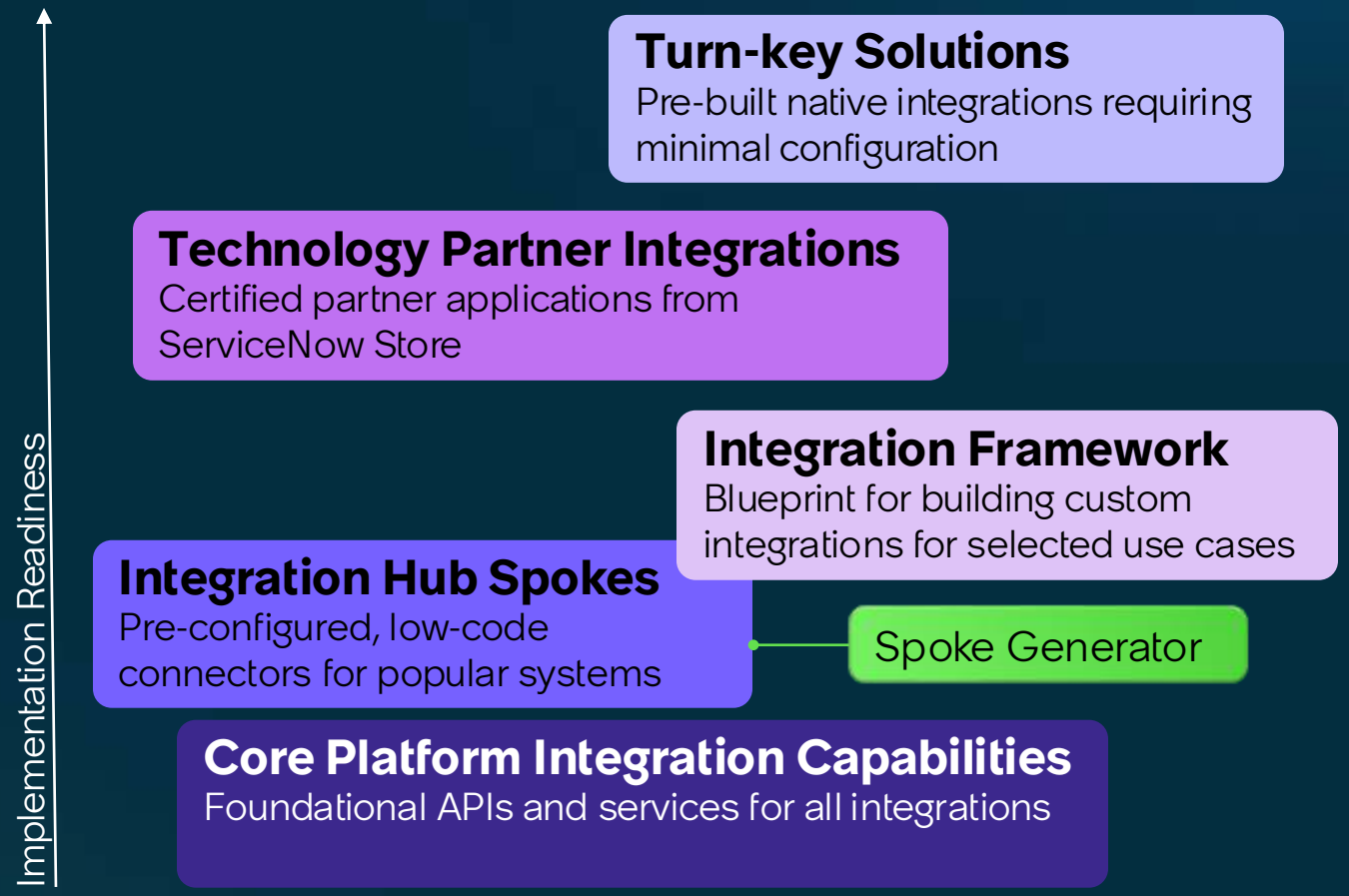
ServiceNow Integration Capabilities

The OOTB-First Integration Strategy

Don't build what already exists

Leverage ServiceNow's investment in pre-built integrations and capabilities

- Reduced development time and cost
- Lower maintenance burden
- Automatic updates with releases
- Proven reliability and support
- Faster time-to-value

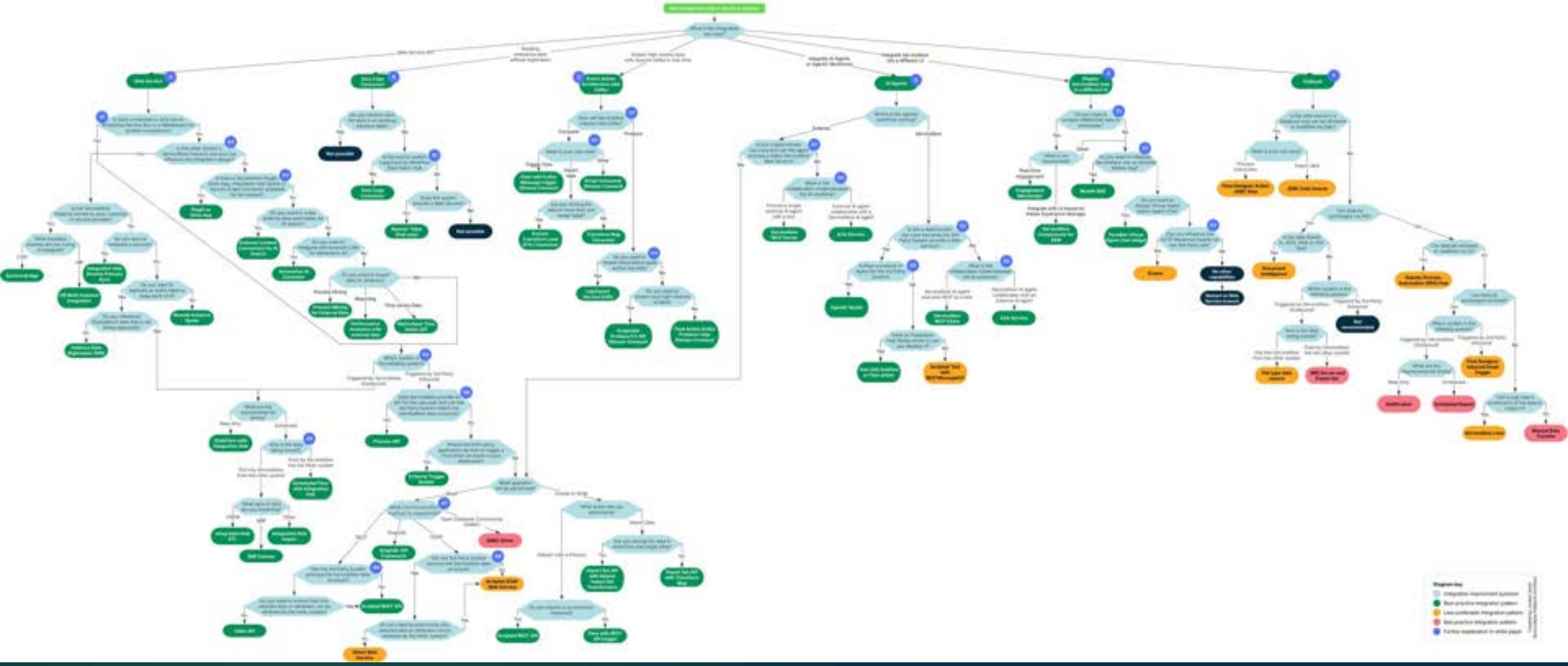


The integration labyrinth

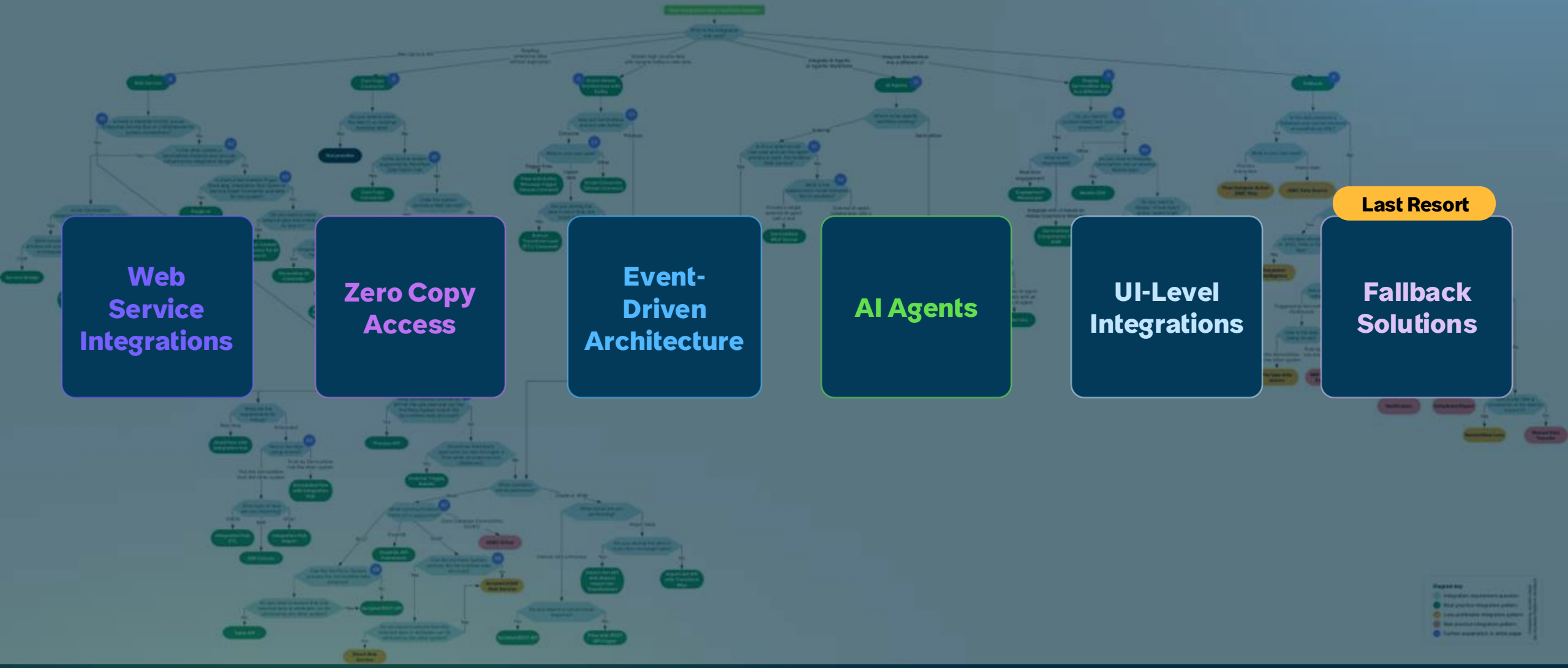
Too many choices, too little guidance

- 50+ integration solutions, capabilities, or building blocks
- Various patterns and frameworks
- Multiple protocols

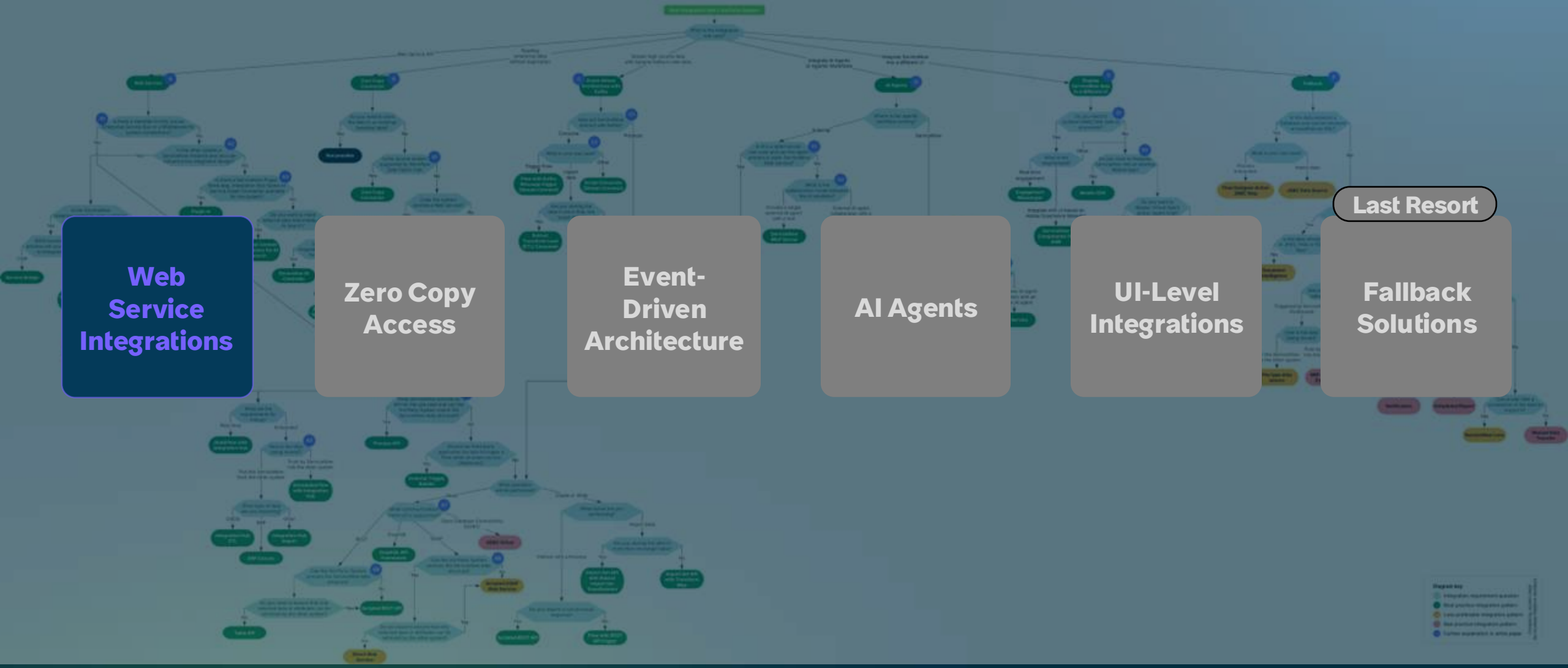
ServiceNow Integration Pattern Decision Tree



ServiceNow Integration Pattern Decision Tree



ServiceNow Integration Pattern Decision Tree



Web Services

API-Based System Connectivity

When to Use:

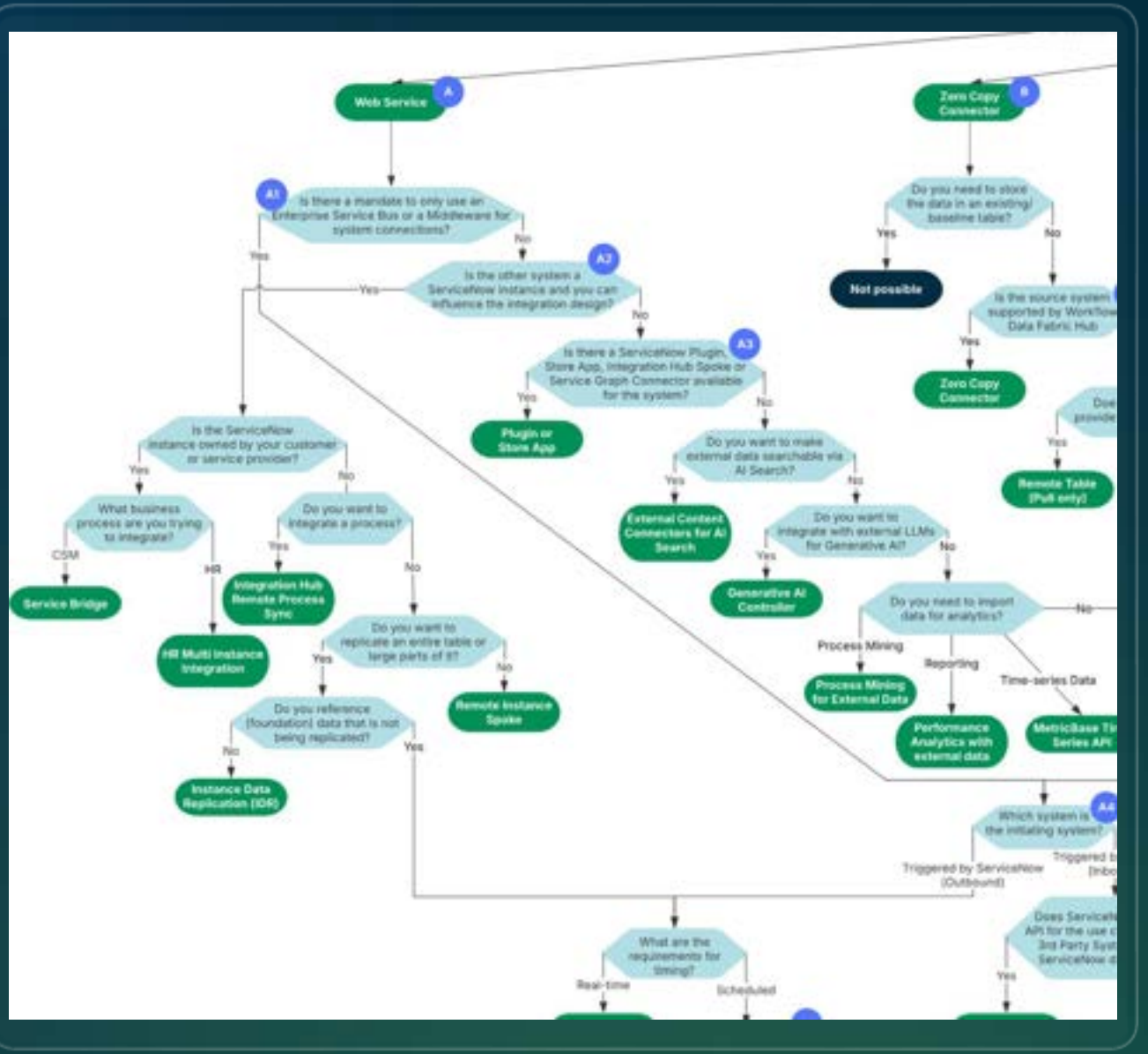
- Standard system-to-system integrations (the preferred default)
- Industry-standard protocols available
- Most common integration scenario that handles 80% of use cases

Solutions:

- REST, GraphQL, SOAP
- Integration Hub, Platform APIs, Scripting and may more

Key Limitation:

- Web services excel at moderate-volume, structured data exchange where data can be persisted in ServiceNow.



Key Decisions: Middleware

Evaluating the Middleware Layer

Pros

- Centralized governance and monitoring
- Reuse existing middleware investment
- Single security/compliance checkpoint
- Enterprise architecture alignment
- Cross-system transformation capabilities

Cons

- Integration Hub Spokes need direct API access
- API misalignment requires customization
- Additional failure points and latency
- Higher TCO and maintenance burden
- ITOM tools require MID Server instead

Decision Framework:

Use middleware when

- Enterprise mandate exists, established infrastructure, centralized governance critical

Direct integration when

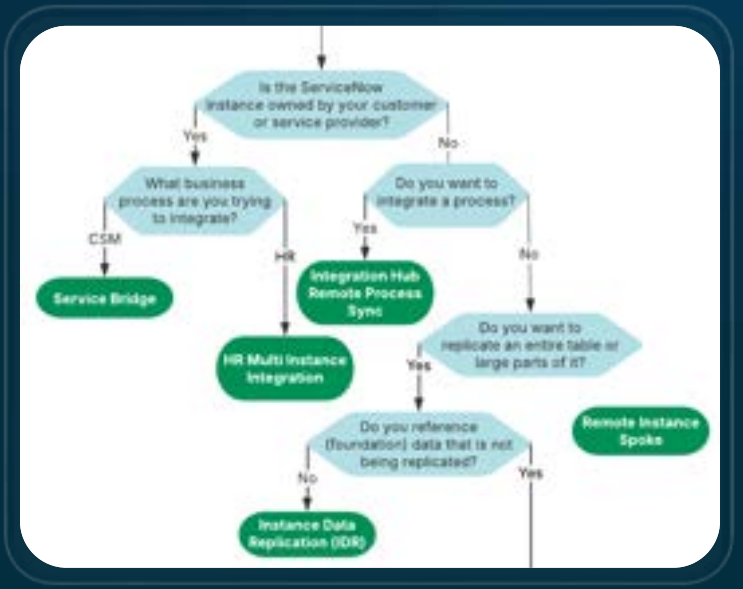
- No organizational requirement, performance critical, ITOM solutions needed

Middleware adds value for enterprise governance but increases complexity ➤ evaluate trade-offs carefully

Key Decisions: Instance Integrations

Specialized Patterns for Multi-Instance Architectures

	Service Bridge	Instance Data Replication (IDR)	Remote Process Sync (RPS)	Remote Instance Spoke
Best for	Telco/Tech/MSP with provider-consumer relationship	Real-time (bi-directional) replication of large datasets with just configuration	Task workflows with guaranteed order execution	Simple, occasional integrations
Limitations	Very limited use cases and low flexibility	Basic mapping and transformation capabilities	Not suitable for Non-Task use cases	Not for heavy use; additional logic for advanced use cases required



Recommendation:

- Each solution targets specific use cases
- Avoid multi-instance architecture when single-instance strategy achieves goals

Key Decisions: Integration Design

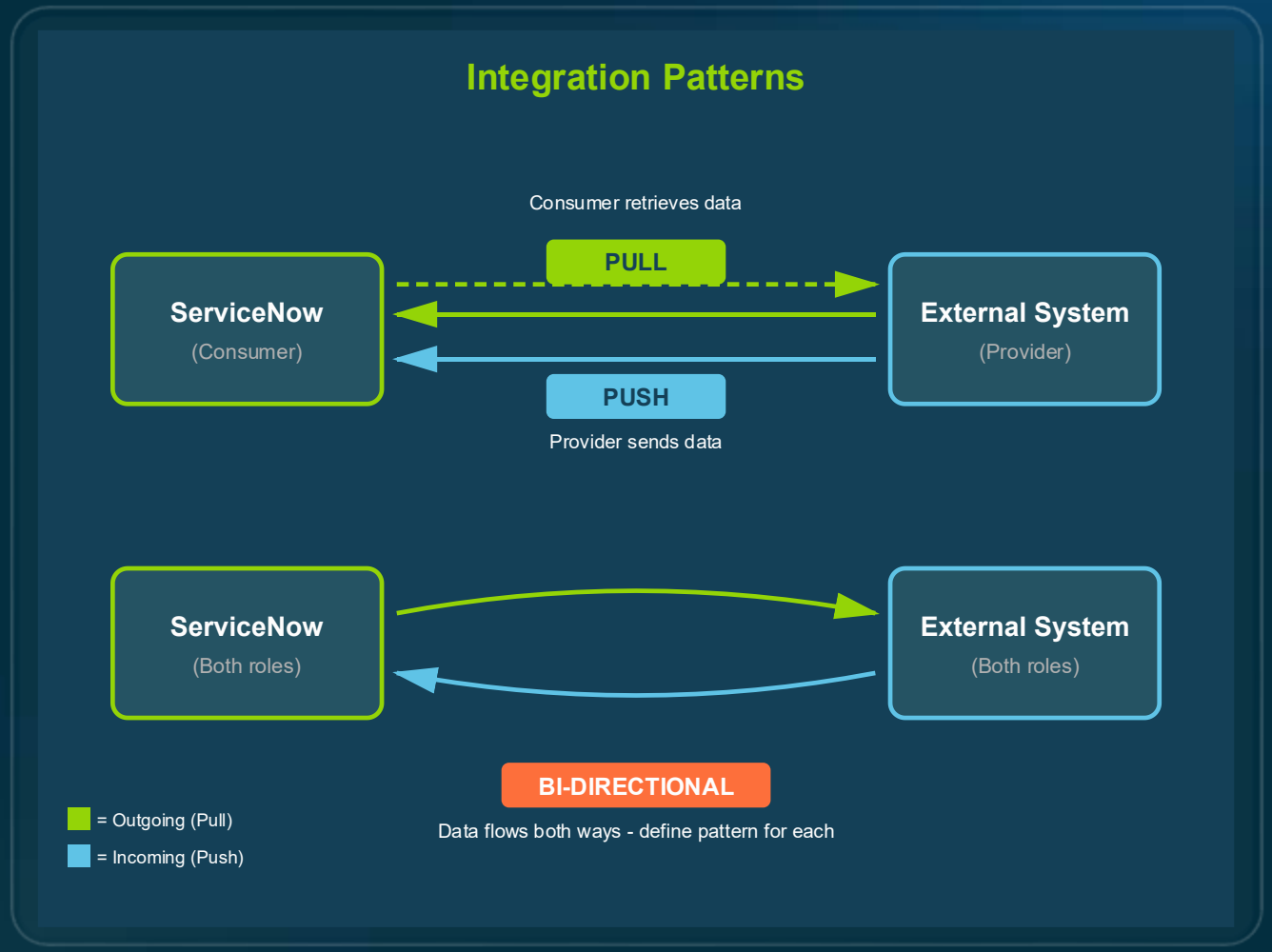
Defining System & Dataflow Direction

Two Critical Decisions:

- 1. Initiating System: Which system triggers the integration process?
- 2. Data Flow Direction (Pull vs. Push)
  - **Pull**: Consumer retrieves data from provider
  - **Push**: Provider sends data to consumer
  - **Bi-directional**: Define a pattern for each direction

Best Practice Guidance:

- No universal "right" answer - depends on control, timing, and capabilities
- Establish a platform-wide standard to ensure consistency across integrations



Key Decisions: Protocols & APIs

Defining System & Dataflow Direction

Protocols (Preference Order):

- REST > GraphQL > SOAP > ODBC

Read Operations (Preference Order):

- Process API > Table API > Scripted REST API

Write Operations:

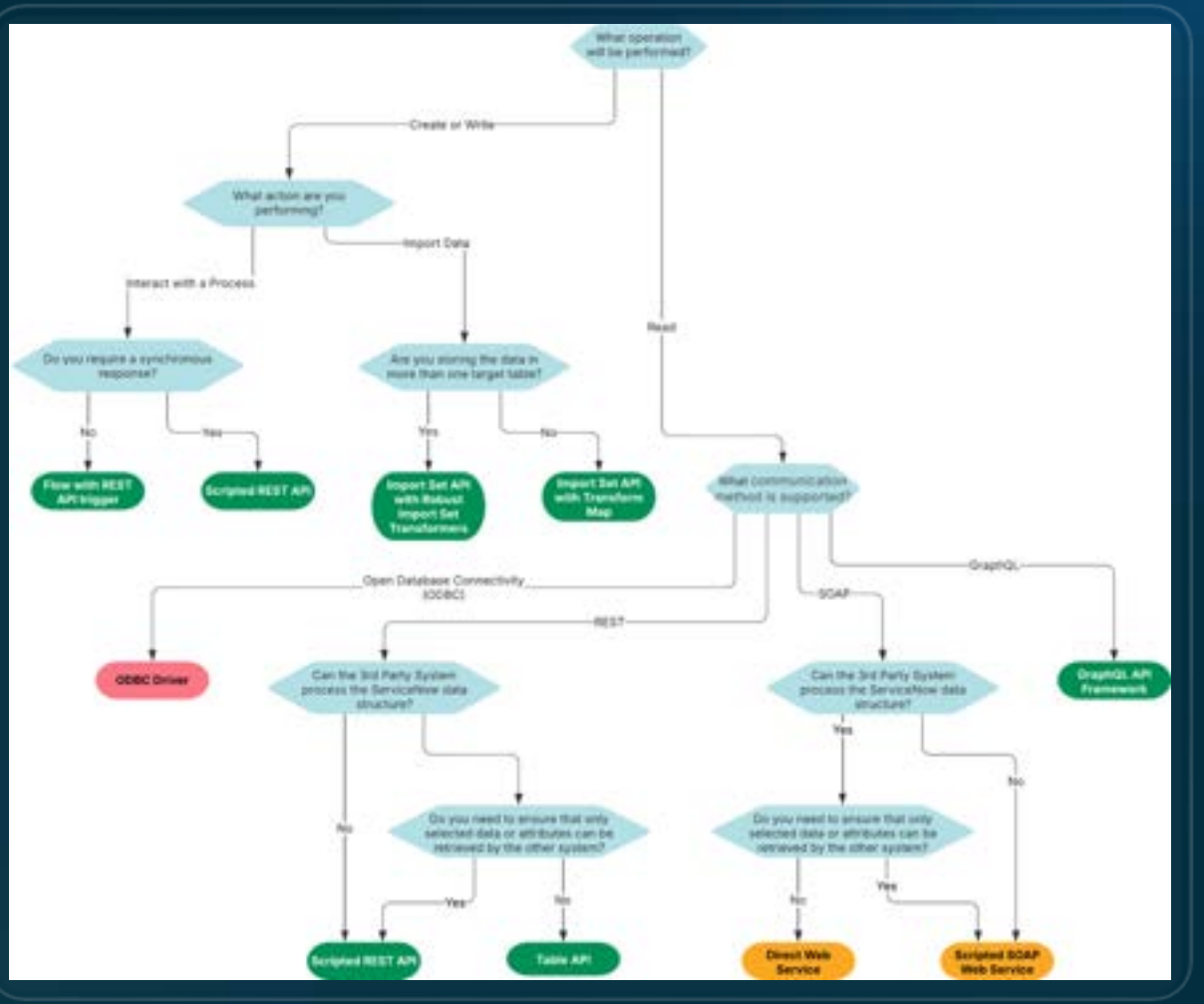
- Import Set API > Scripted REST API > Direct Table API writes (Avoid)

Data Format:

- JSON > XML

Key Guidance:

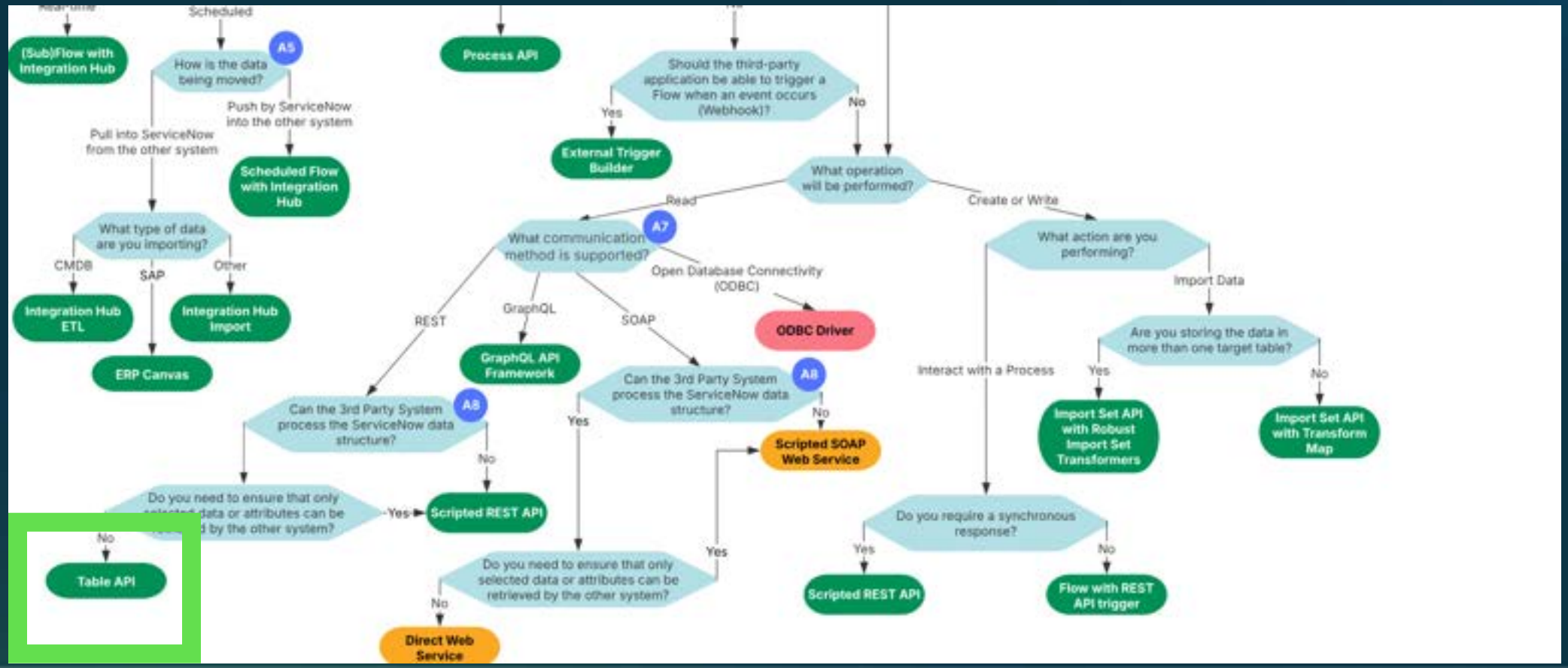
- Stay within REST + JSON for simplicity and maintainability



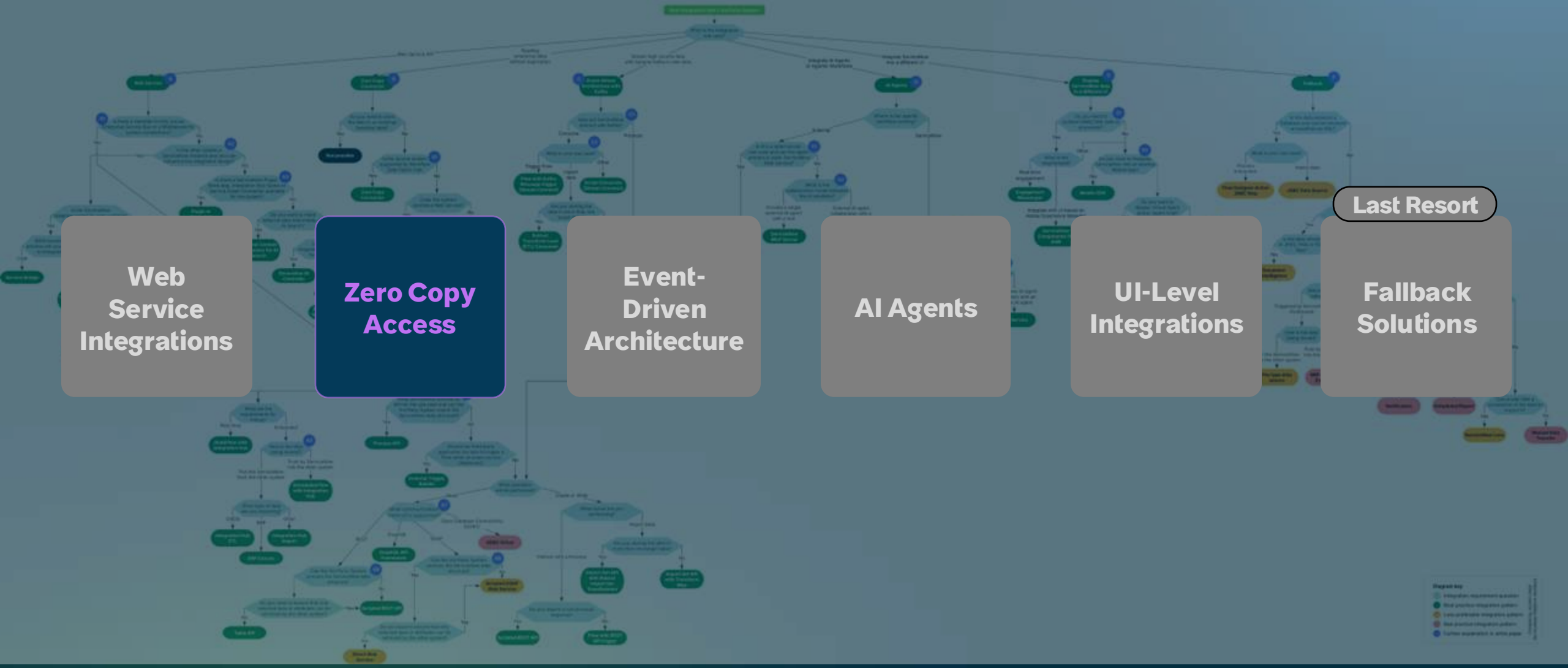
Example: Wind Turbine Work Order Retrieval

An energy company with over 800 turbines. Real-time turbine status is displayed in a specialized tool, "TurbineView," located at the operations center. Active ServiceNow work orders must be visible within TurbineView to connect maintenance activities with performance.

Example: Wind Turbine Work Order Retrieval



ServiceNow Integration Pattern Decision Tree



Web Service Integrations

Zero Copy Access

Event-Driven Architecture

AI Agents

UI-Level Integrations

Last Resort

Fallback Solutions

Zero Copy

Data Residency

When to Keep Data External:

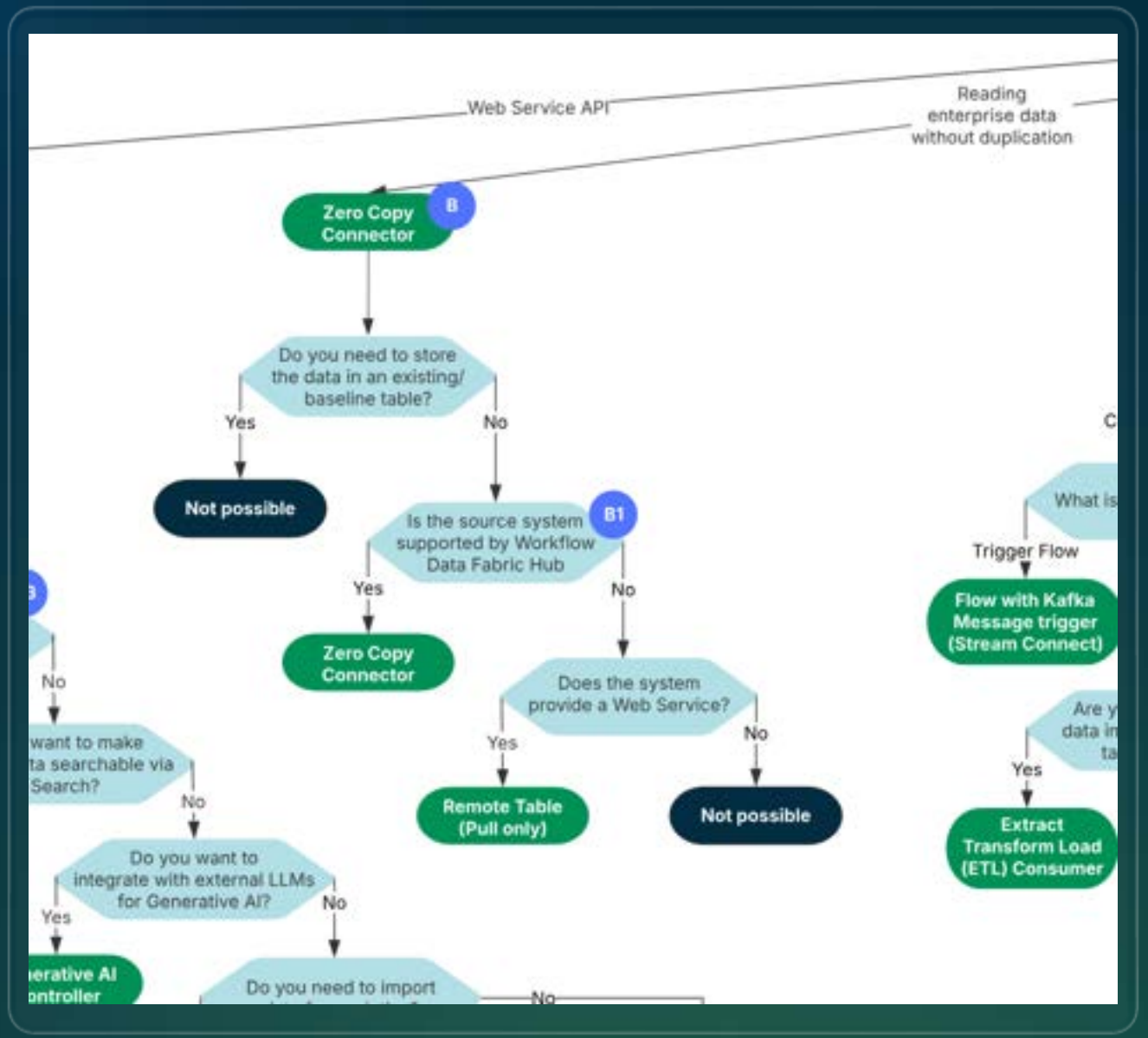
- **High-volume or high-velocity data** that impacts platform performance
- **Compliance constraints** (GDPR, HIPAA, PCI DSS, etc.) requiring data sovereignty

Solutions:

- **Zero Copy Connectors**: Access external data without duplication (preferred)
- **Remote Tables**: Fallback when Zero Copy unavailable

Key Limitation:

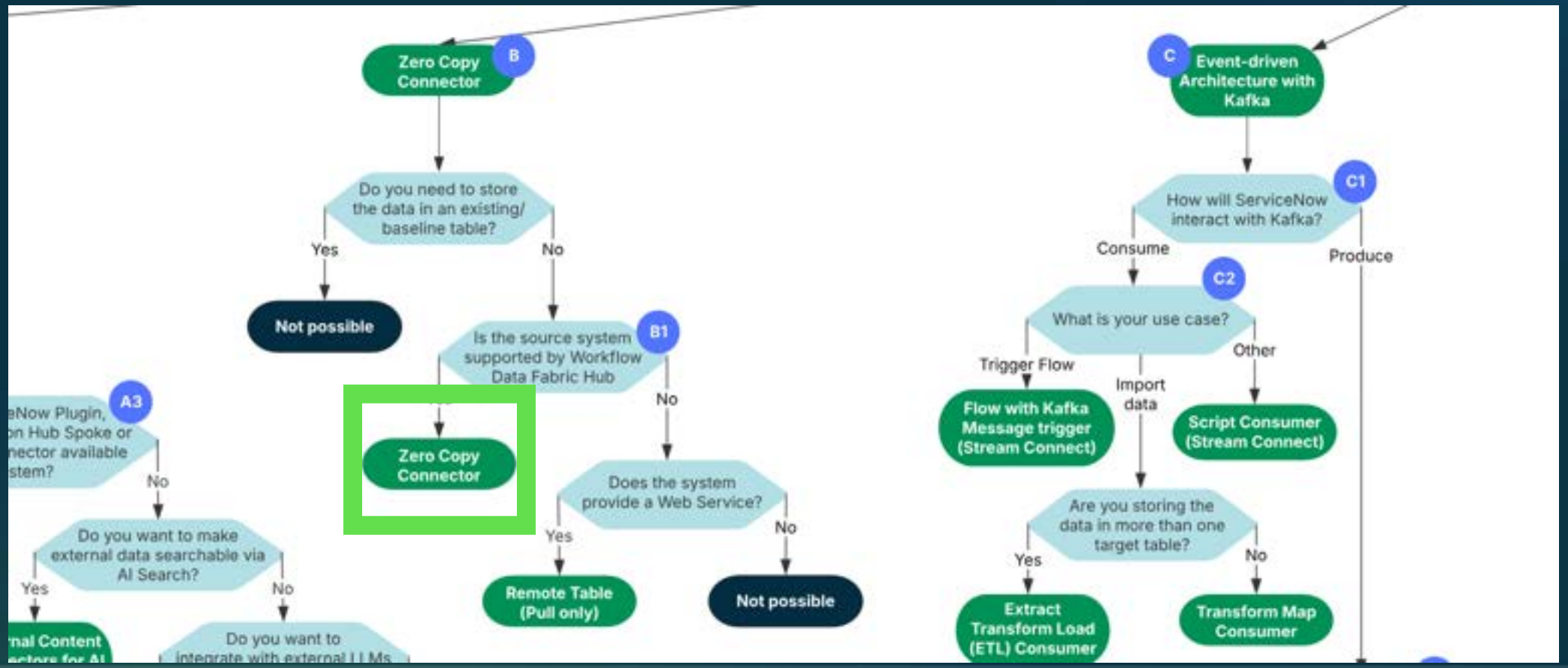
- Only work for new tables - cannot replace existing ServiceNow structures (e.g. CMDB, CSM)



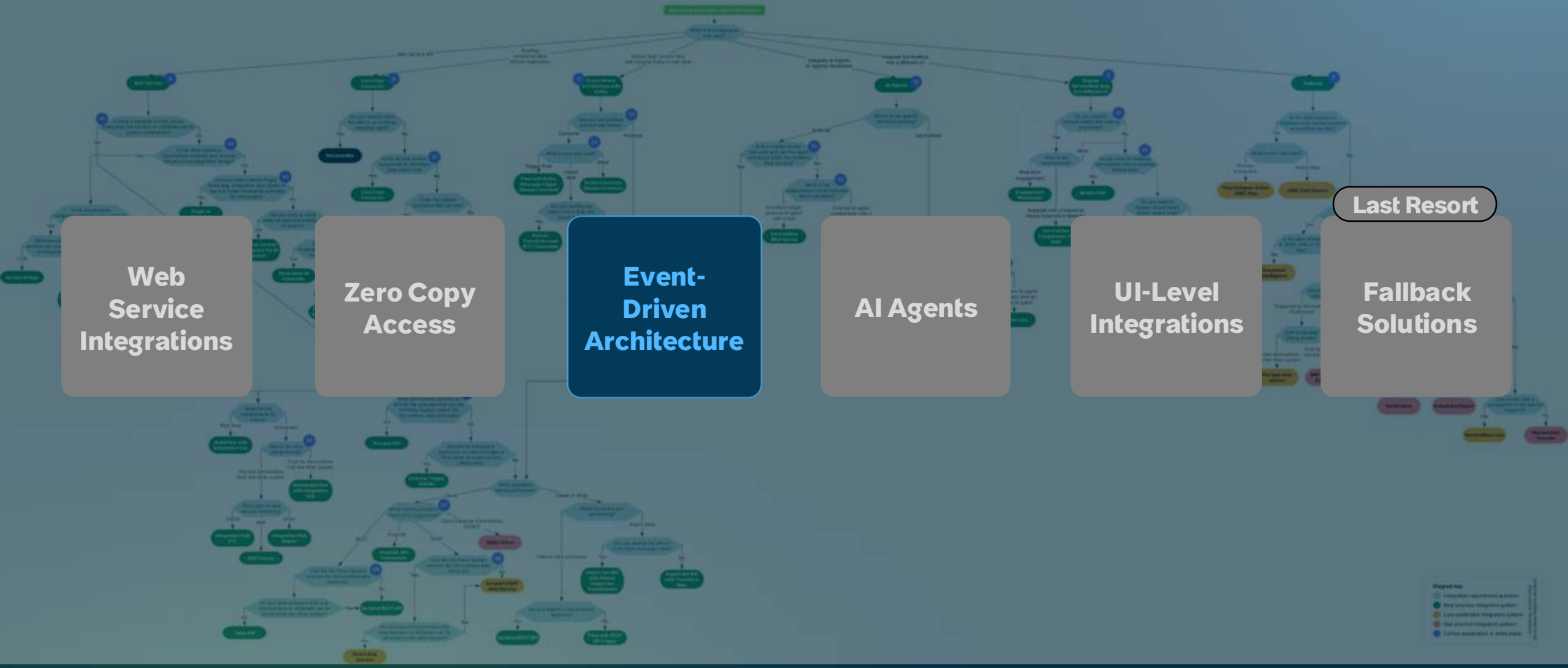
Example: Customer 360

A telecommunications company surfaces 50 million customer records from their data warehouse directly in the CSM Workspace. Support agents see complete customer history, usage patterns, and billing data alongside cases without replicating terabytes of data into ServiceNow or waiting for API calls to return.

Example: Customer 360



ServiceNow Integration Pattern Decision Tree



Stream Connect

Event-Driven Architecture

When to Use:

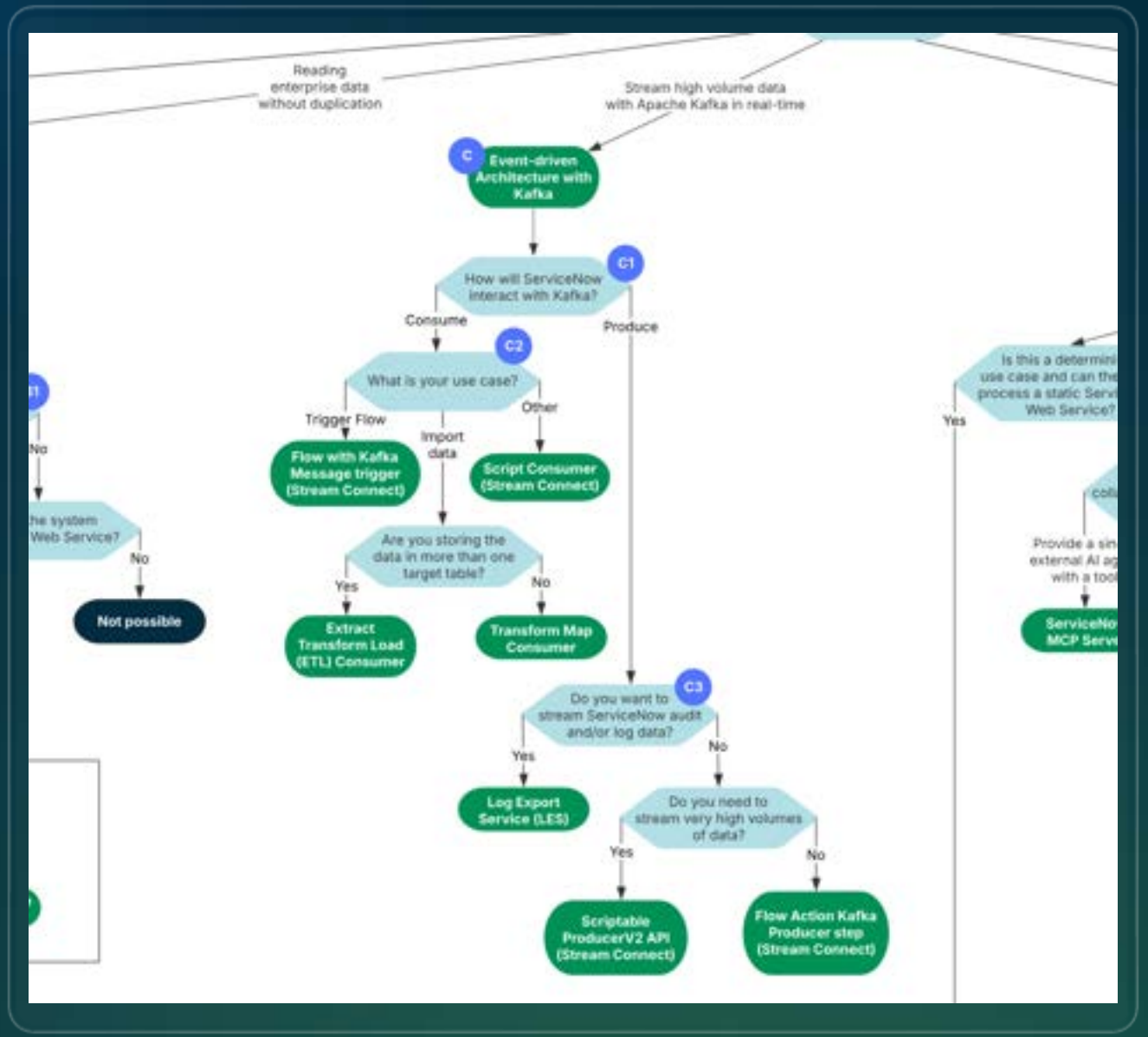
- **Real-time event processing** required across multiple systems
- **High-volume streaming data** (IoT sensors, logs, transactions)
- **Decoupled system communication** for improved scalability and reliability

Solutions:

- **Stream Connect**: Bidirectional Kafka integration
- Log Export Service: Stream selected ServiceNow log tables to Kafka

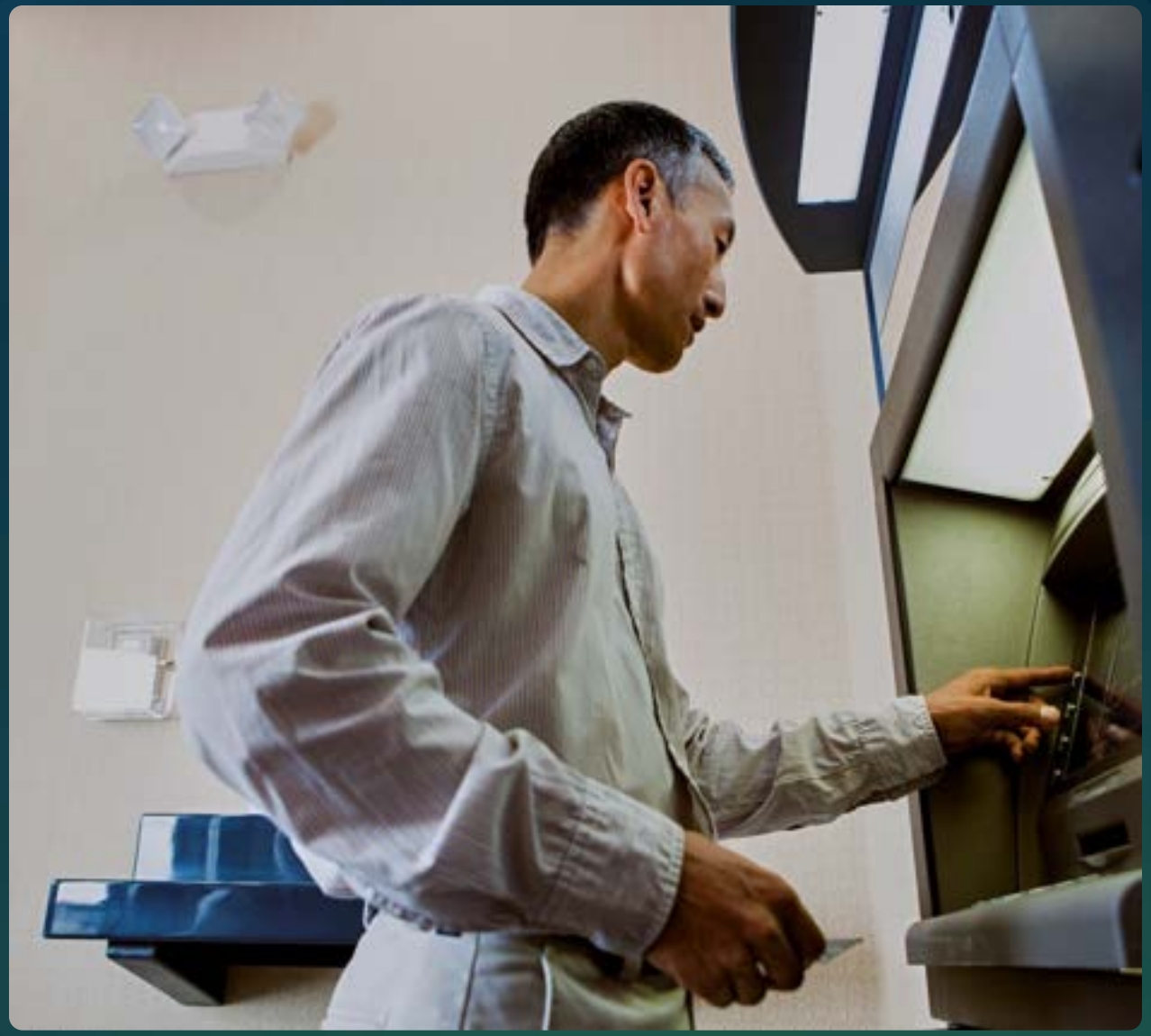
Key Limitation:

- Non-Kafka architectures

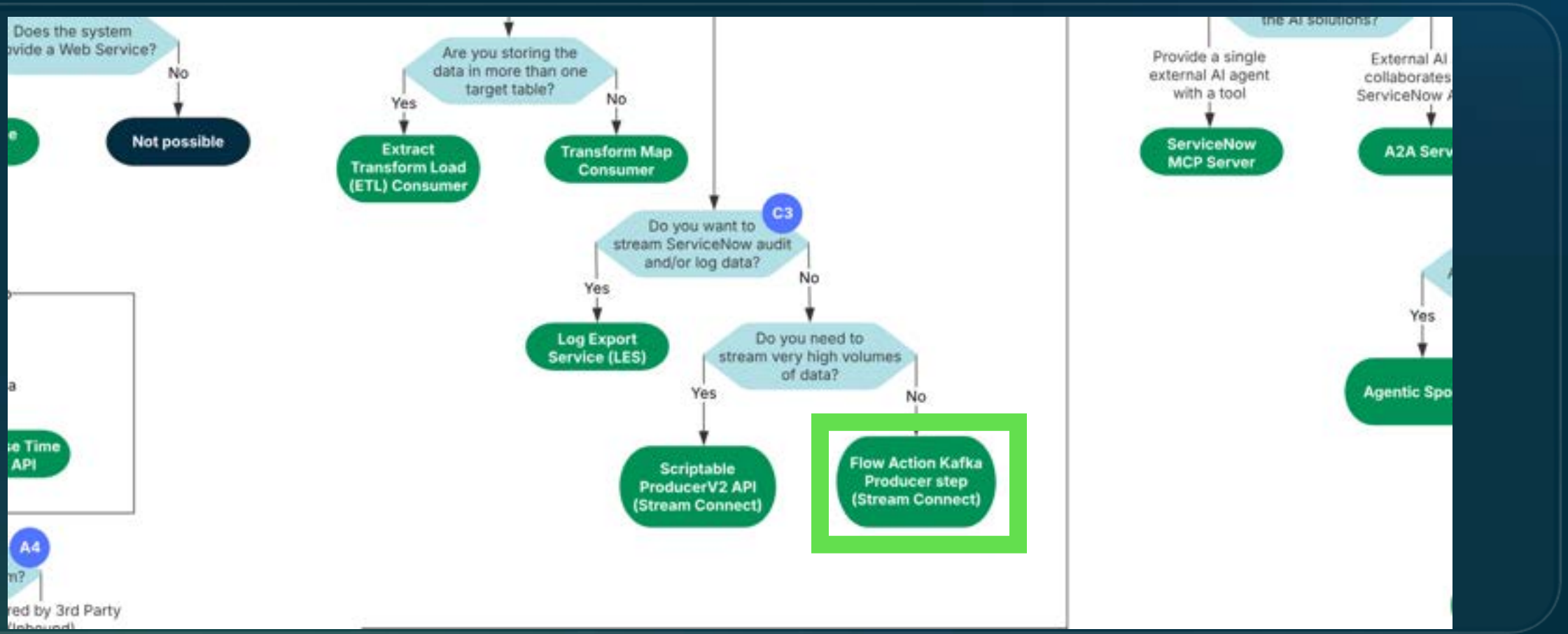


Example: Banking Account Journey

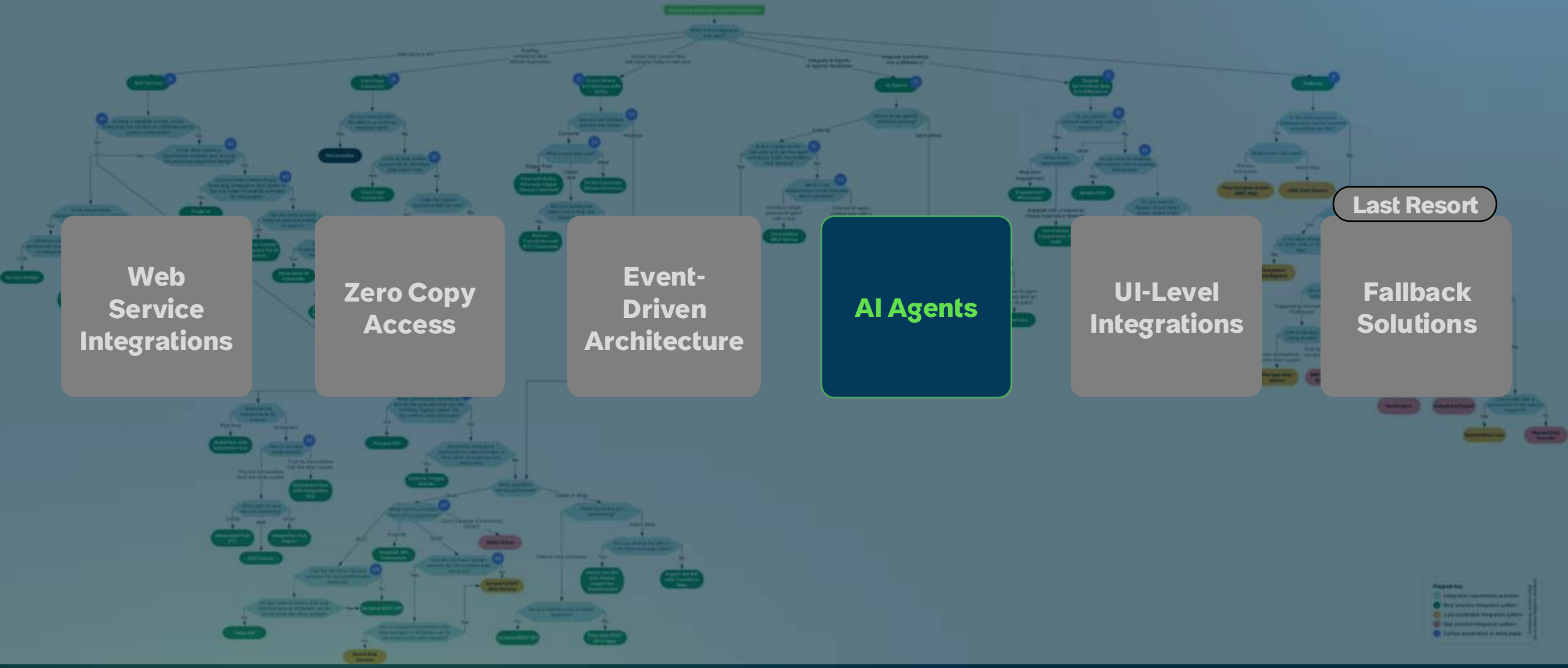
A retail bank publishes new account events to their customer engagement platform. External journey orchestration triggers welcome sequences, financial wellness tips, and cross-sell offers based on real-time transaction behavior - coordinating personalized outreach across channels requiring dedicated marketing infrastructure.



Example: Banking Account Journey



ServiceNow Integration Pattern Decision Tree



AI Agents

Agentic Workflow Patterns

When to Use:

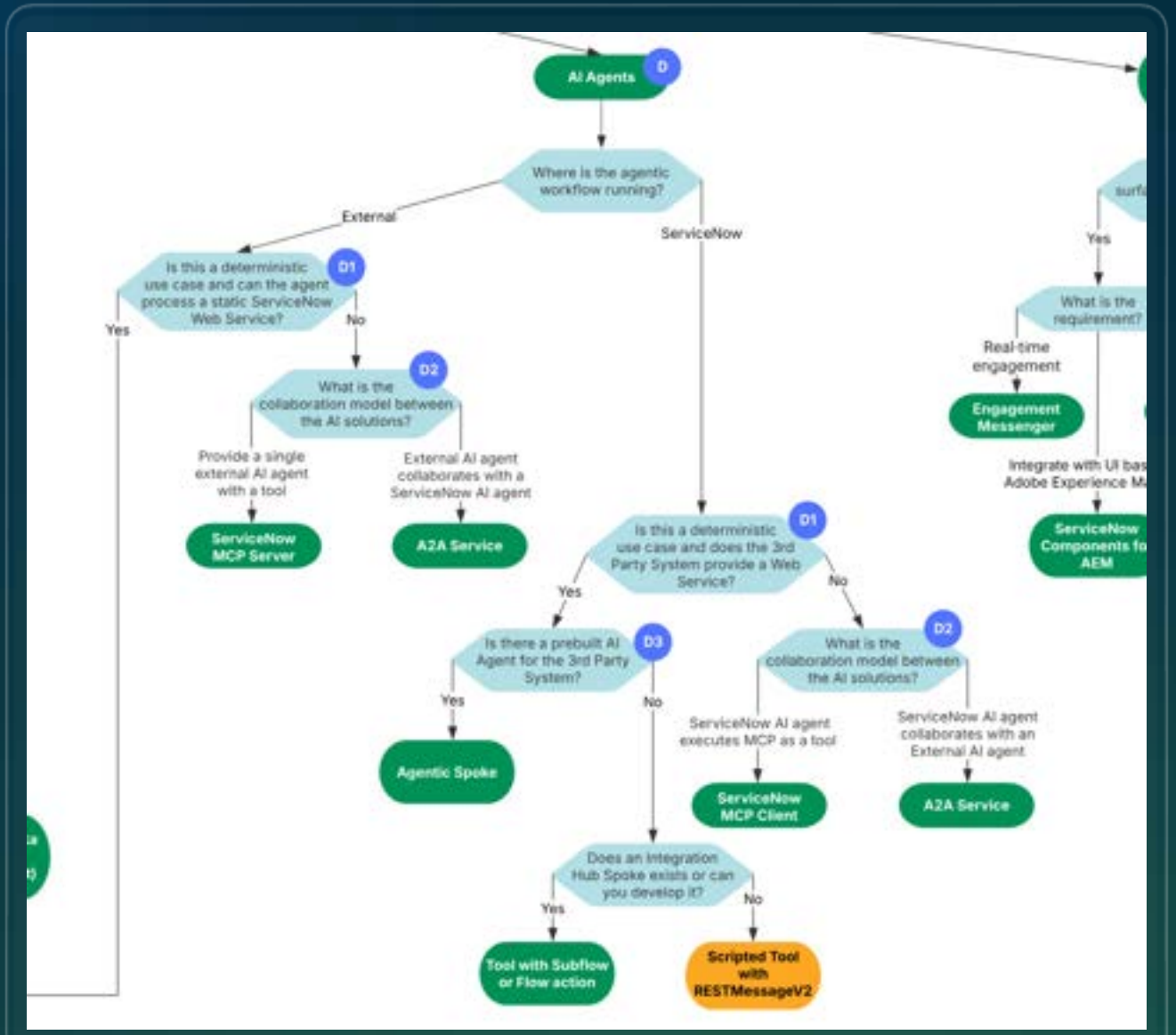
- AI agents require dynamic, flexible integrations rather than rigid workflows
- Multi-agent coordination across platforms
- External AI agents need ServiceNow capabilities

Solutions:

- **MCP (Model Context Protocol)**: Dynamic tool discovery for single agent extension
- **A2A (**Agent2Agent): Multi-agent collaboration and coordination
- **Agentic Spokes & Tools**

Key Limitation:

- Can create variable outcomes, may not work for workflows requiring predictability



AI Agents

Choosing the Right Pattern for Agentic Workflows

	Agentic Spokes	A2A Agent-to-Agent	REST/Web Services	MCP Model Context Protocol
Use Case	Pre-built agent integrations for third-party systems	Multi-agent coordination, cross-platform dialogue	Predictable schemas, auditability, compliance-ready	Dynamic tool discovery, runtime capability extension
Best for	When OOTB solution exists	Complex workflows requiring autonomous agent collaboration across systems	Regulated workflows or business critical processes	Single agent extension, flexibility valued
Pros	• Hundreds of systems supported OOTB • Low maintenance • Modular and reusable	• Enables agent modularity • Multi-turn coordination • Autonomous task delegation.	• Simple CRUD operations • Well-documented • Deterministic and secure	• Purpose-built for agentic AI • Prompt-driven tool selection • No hard-coded parameters
Cons	• Limited to available spokes • Hard-coded parameters	• Newer protocol, higher complexity • Requires 3rd-Party-System AI agents • Less performant	• Custom-built • Hard-coded parameters	• Newer standard, evolving • Less deterministic • Subject to LLM variability

AI Agents

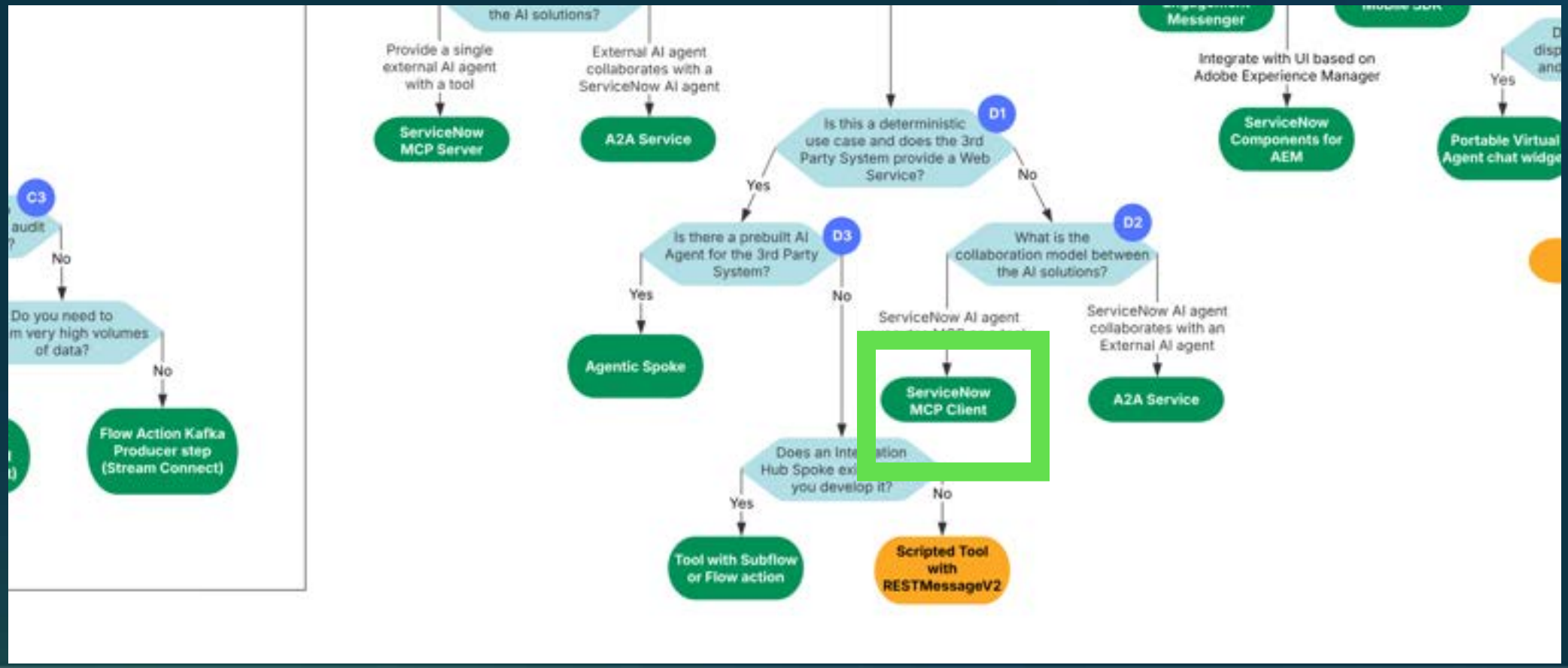
Added as Tools to AI Agents

servicenow

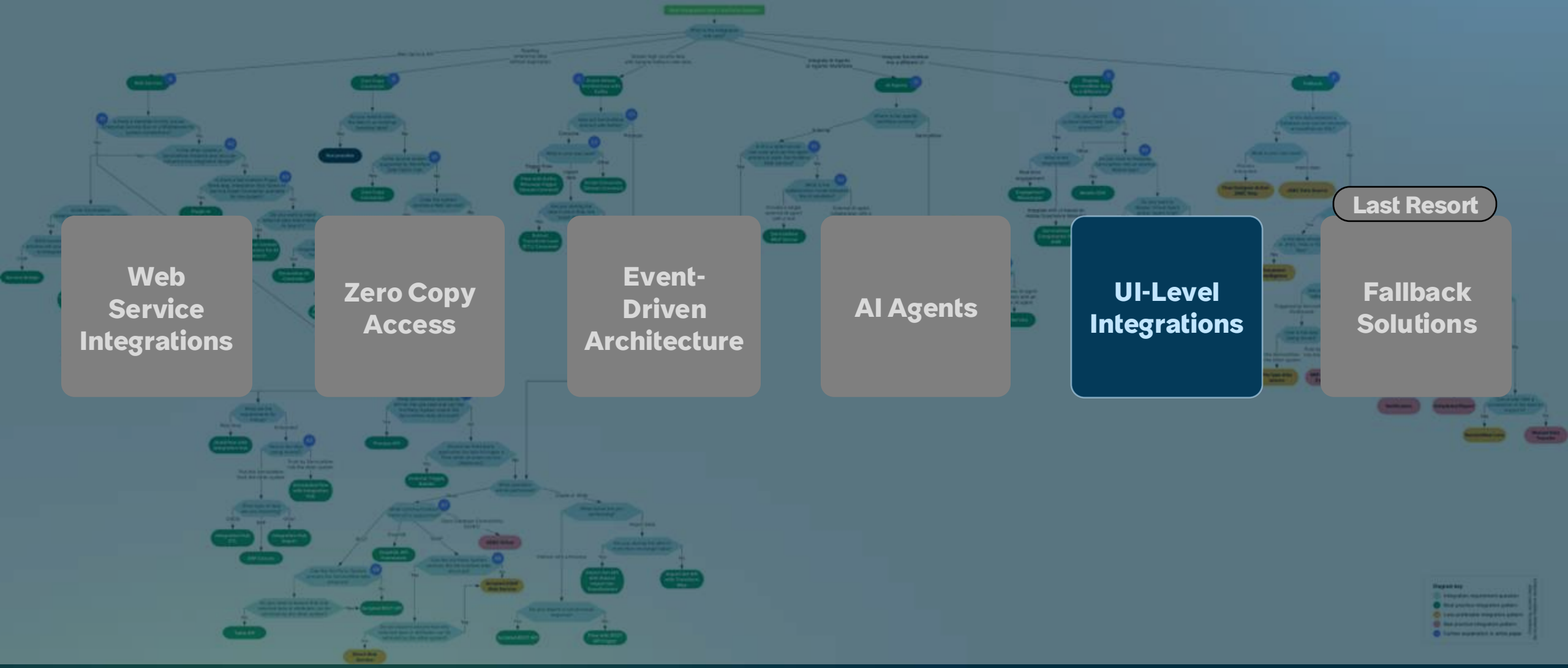
Example: AI-Powered Contract Risk Assessment

A professional services firm enables business users to assess contract risks via their ServiceNow Legal Agent, where questions like "Can we accept this vendor's liability cap?" dynamically trigger clause extraction, policy comparison, and risk scoring using a 3rd-Party Contract Lifecycle software - eliminating legal bottlenecks for routine contracts.

Example: Contract Risk Assessment



ServiceNow Integration Pattern Decision Tree



Web Service Integrations

Zero Copy Access

Event-Driven Architecture

AI Agents

UI-Level Integrations

Last Resort

Fallback Solutions

UI-Level Integrations

Embedding ServiceNow Experiences

When to Use:

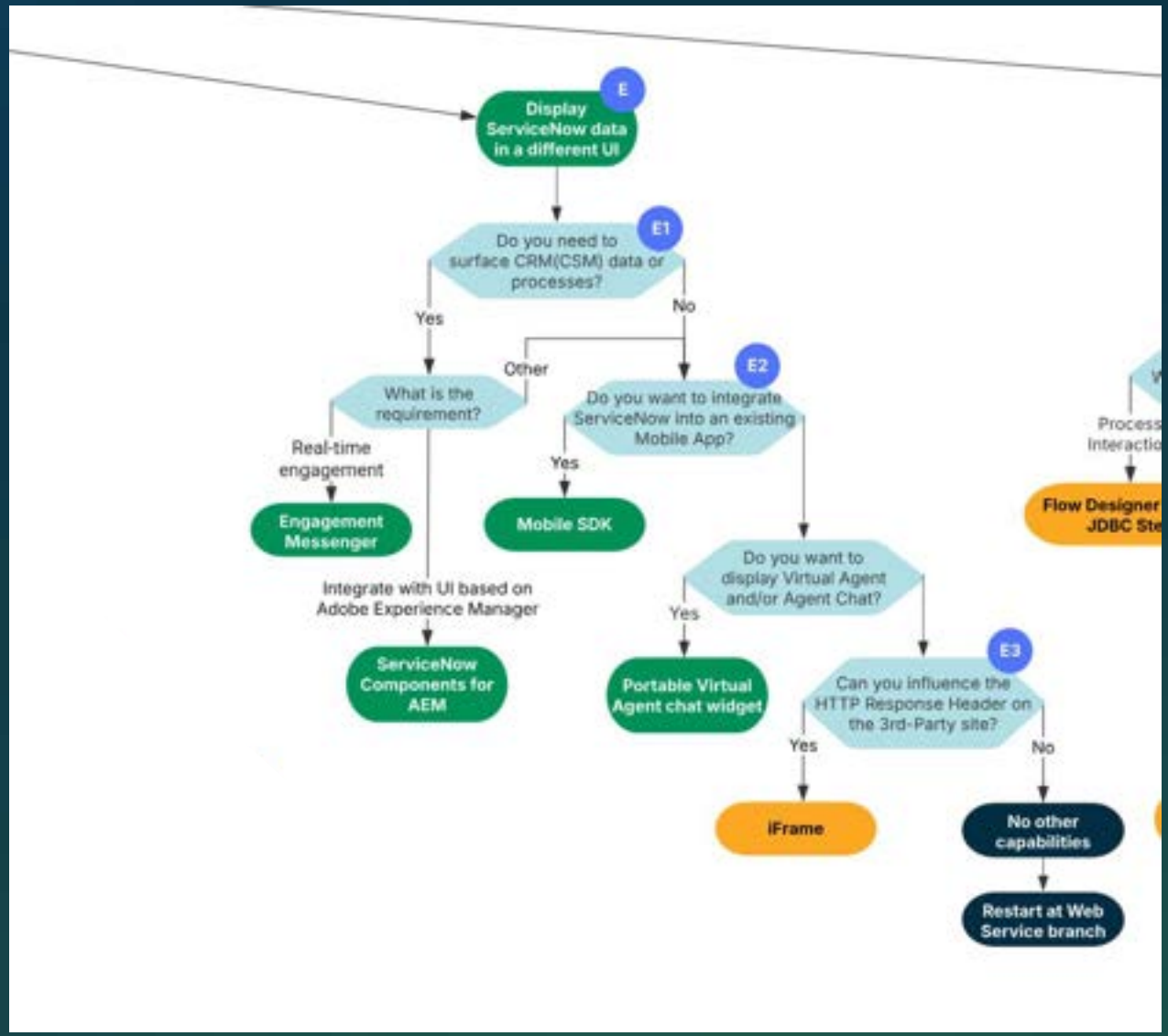
- Display ServiceNow within external UIs
- Provide a consistent user experience across multiple systems

Solutions:

- **Engagement Messenger**
- Service **Mobile SDK**
- **ServiceNow Components for AEM**
- **Portable Virtual Agent Widget**
- **iFrame**: Basic embedding (use as last resort)

Key Limitation:

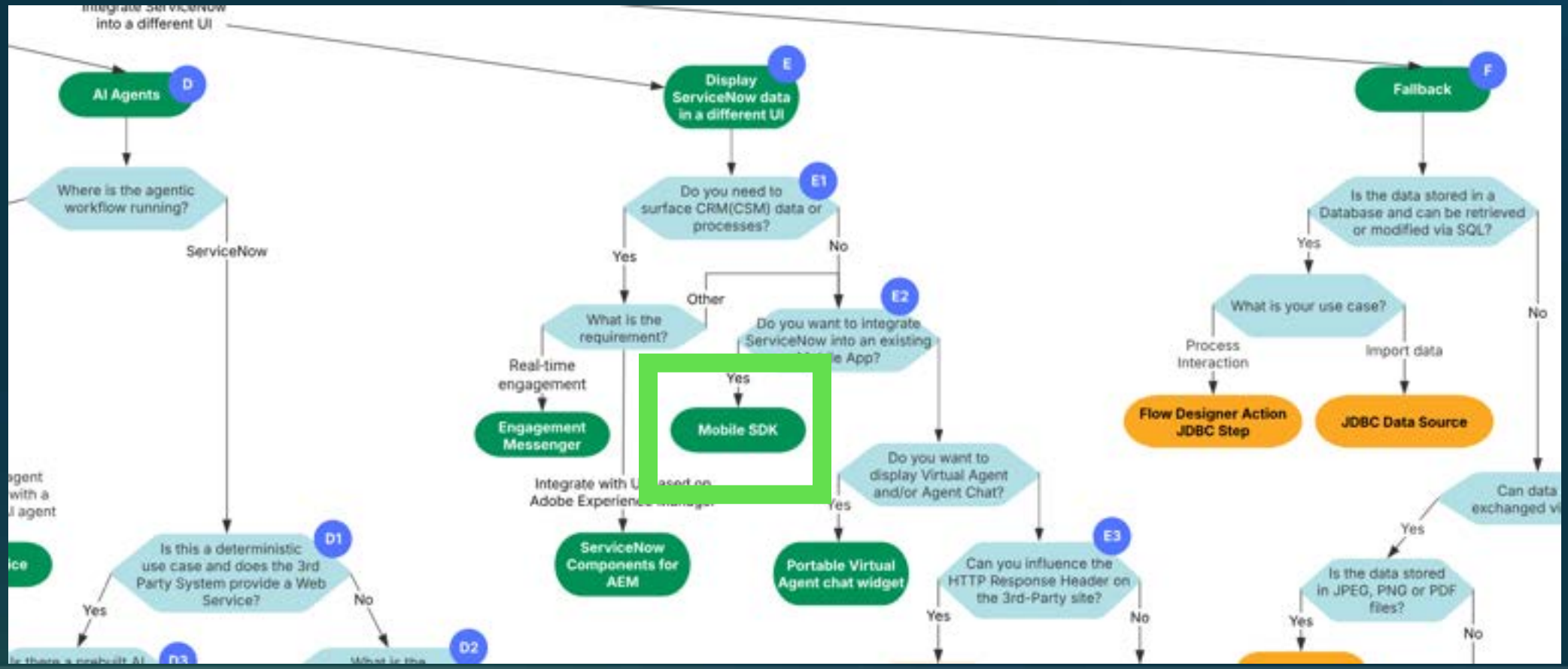
- Limited to display/interaction: use web services for data sync



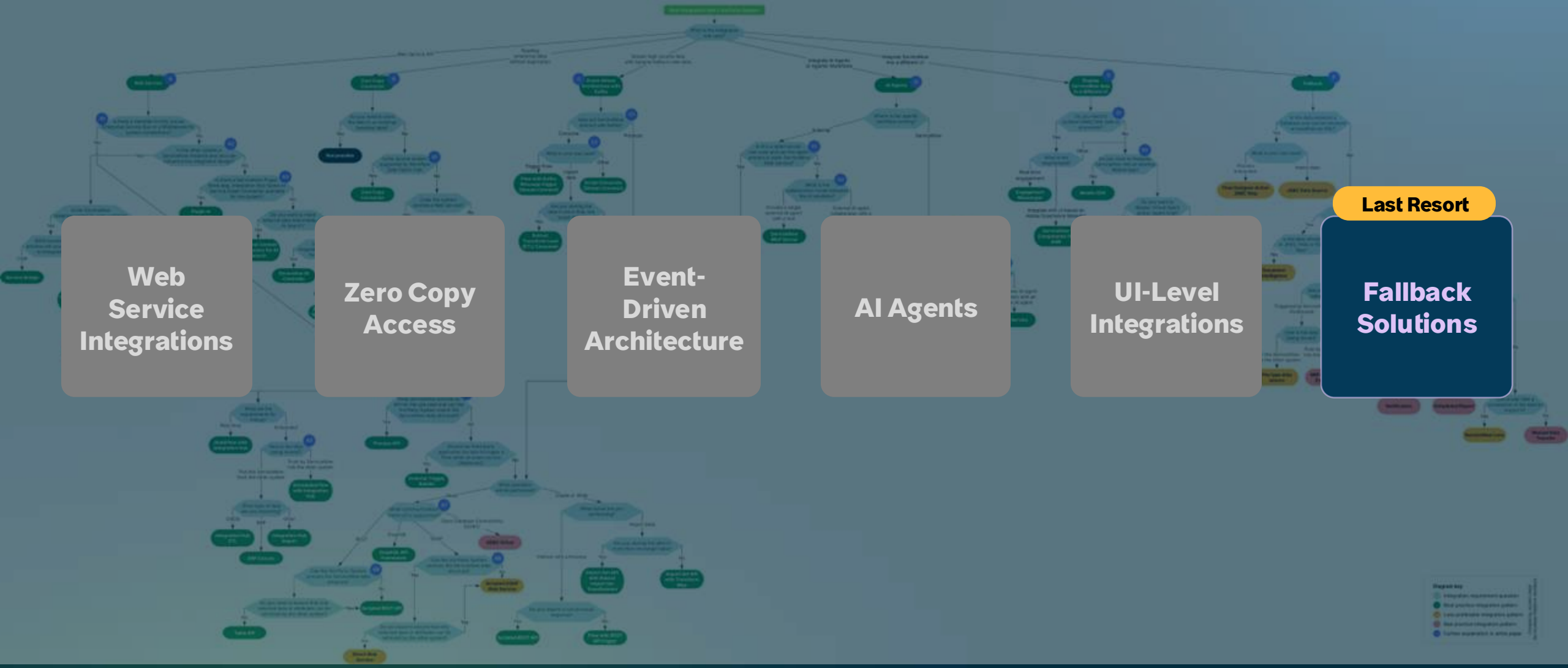
Example: Airline Crew App with Inflight Service Requests

A global airline integrates ServiceNow features into their current crew management application. Flight attendants can file expense reports, ask for schedule adjustments, and report equipment problems while in flight, with requests queued for handling once the plane lands.

Example: Contract Risk Assessment



ServiceNow Integration Pattern Decision Tree



Web Service Integrations

Zero Copy Access

Event-Driven Architecture

AI Agents

UI-Level Integrations

Last Resort

Fallback Solutions

Fallback

Last Resort Integration Patterns

When to Use:

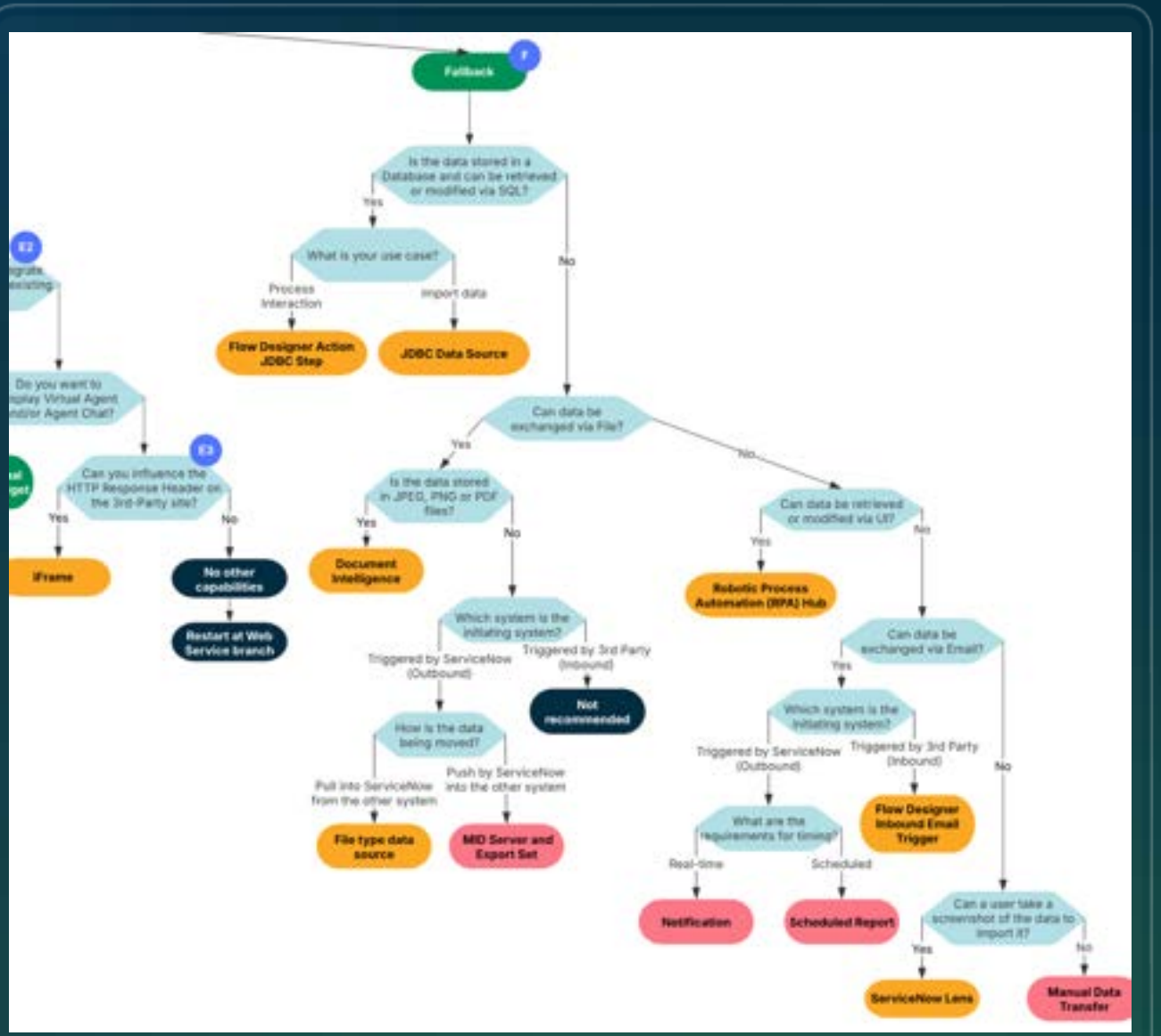
- Legacy systems or technical constraints prevent web service capabilities
- Interim solutions during system modernization

Solutions:

- File-based integrations
- Robotic Process Automation
- Email
- MID Server Scripts
- JDBC

Key Limitation:

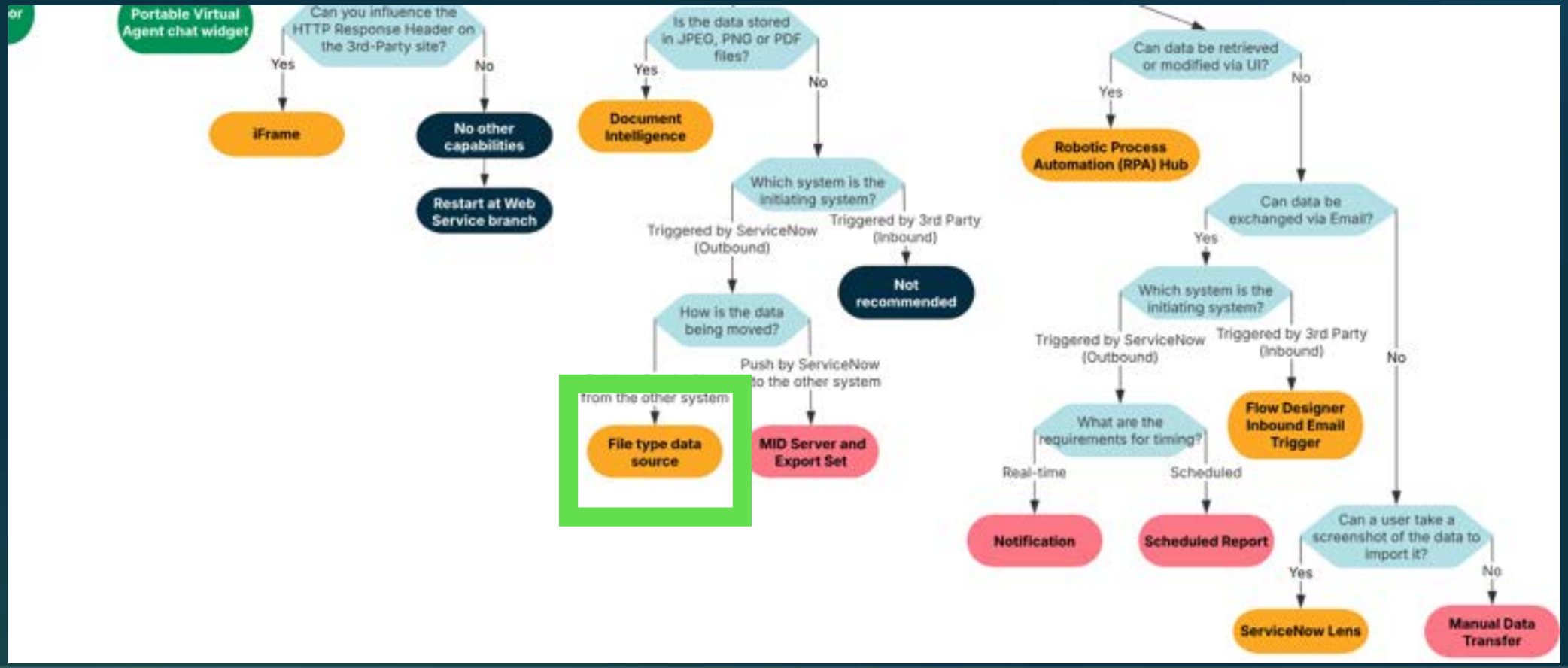
- High maintenance, poor scalability, security risks



Example: Benefits Data from Legacy HR System

A government agency imports employee benefits enrollments from their 30-year-old mainframe HR system. Nightly COBOL batch jobs generate CSV-files dropped to a secure FTP location. The only interface this system will ever support without a multi-million dollar modernization effort.

Example: Benefits Data from Legacy System



CONNECT
to any system with diverse forms of connectivity

Web Service Integrations

Best for

Standard system-to-system connectivity via APIs

Zero Copy Access

Best for

High-volume data or compliance constraints

Event-Driven Architecture

Best for

Real-time event streaming and processing with Apache Kafka

AI Agents

Best for

Flexible AI agent coordination workflows

UI-Level Integrations

Best for

Embedding ServiceNow in external application UIs

Last Resort

Fallback Solutions

Best for

Legacy systems when everything else fails

Governance

Integration Governance

Strategic Alignment

- How does this transform our business?

Security & Compliance

- How do we comply with security policies and regulatory requirements?

Performance & Scalability

- What current and future workloads are required?

Enterprise Architecture Alignment

- How does this align with the larger IT landscape?

Platform Technical Governance

- How do we ensure ServiceNow Best Practices are used?

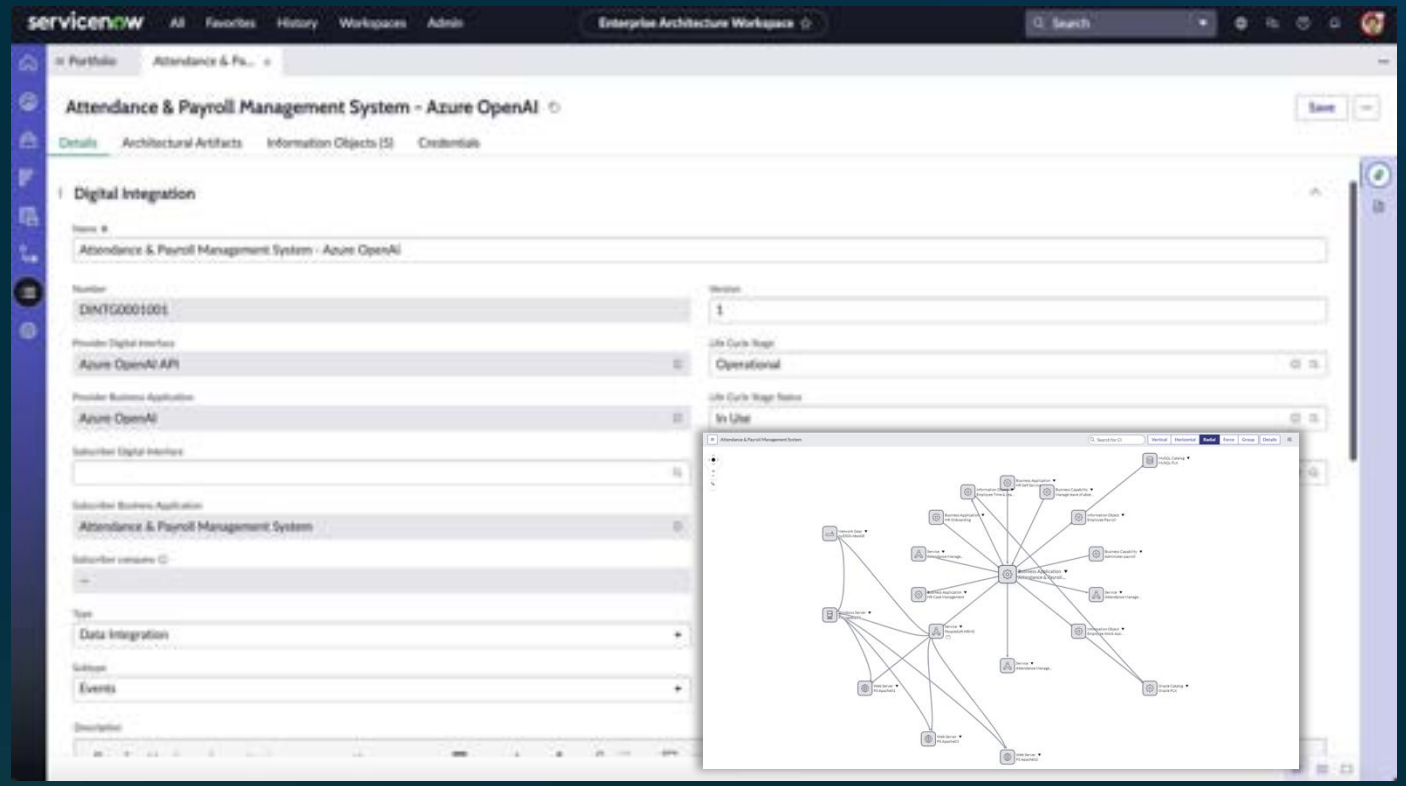
Operations

- How do we "run" this together with our counterpart system?

Digital Integration Management

Effectively manage APIs by leveraging models to visualize and understand app integration landscape

- Visualize dependencies in the integration landscape
- Standardize integration intake forms
- Workflows to ensure alignment with business needs



Key takeaways

Key Recommendations

OOTB first, always

Check native integrations, spokes, and frameworks before custom development

1

Establish platform-wide standards

Consistency prevents technical debt

2

Document your integrations

A labyrinth is less scary with a map

3

Reserve a Refactor Budget

Review less optimal patterns for refactoring opportunities

4

Thank you

servicenow®

Q&A

servicenow®